```
/**
 * supt-resonance.js — JavaScript reference port of the frozen SUPT probe.
 *
 * Ported from supt_resonance.py. alpha = 0.01, frozen 2025-01-20. Do not tune.
 *
 * VERIFY THE PORT BEFORE WIRING IT TO ANYTHING.
 * Run `node supt-resonance.js` — it self-tests against reference vectors
 * generated by the Python implementation. All five must pass to 1e-12.
 * If they do not, the port is wrong and any live score it produces is wrong.
 *
 * The five places a port of this operator usually breaks are marked [PORT NOTE].
 */

const ALPHA = 0.01;

// ---------------------------------------------------------------------
// [PORT NOTE 1] median must average the two middle values for even N.
// numpy does this; a naive "sort and take middle" does not, and the whole
// pipeline is downstream of two nested medians.
// ---------------------------------------------------------------------
function median(arr) {
  const a = Float64Array.from(arr).sort();
  const n = a.length;
  if (n === 0) return NaN;
  const mid = n >> 1;
  return n % 2 ? a[mid] : (a[mid - 1] + a[mid]) / 2;
}

// [PORT NOTE 2] MAD is median(|x - median(x)|) — nested, both need the above.
function mad(arr, med) {
  return median(Array.from(arr, (v) => Math.abs(v - med)));
}

function isFiniteNum(v) {
  return typeof v === "number" && Number.isFinite(v);
}

/**
 * The frozen probe. Returns d_ij, or null if the sequence is too short.
 */
export function suptProbe(values) {
  let x = Array.from(values, Number).filter(isFiniteNum);
  if (x.length < 4) return null;
```

```
  const med = median(x);
  const m = mad(x, med);
  x = x.map((v) => (v - med) / (m + 1e-12)); // [PORT NOTE 3] keep every 1e-12

  // cumulative phase, then successive differences
  const phi = new Float64Array(x.length);
  let acc = 0;
  for (let i = 0; i < x.length; i++) { acc += x[i]; phi[i] = acc; }

  const g = new Float64Array(phi.length - 1);
  for (let i = 0; i < g.length; i++) g[i] = phi[i + 1] - phi[i];

  let meanAbs = 0;
  for (let i = 0; i < g.length; i++) meanAbs += Math.abs(g[i]);
  meanAbs /= g.length;
  for (let i = 0; i < g.length; i++) g[i] = g[i] / (meanAbs + 1e-12);

  // EMA cosine accumulator
  const C = new Float64Array(g.length);
  C[0] = Math.cos(2 * Math.PI * g[0]);
  for (let i = 1; i < g.length; i++) {
    C[i] = ALPHA * Math.cos(2 * Math.PI * g[i]) + (1 - ALPHA) * C[i - 1];
  }

  // [PORT NOTE 4] Python's int() truncates. Use Math.floor, never Math.round.
  // If tail > C.length, slice(-tail) returns the whole array — matches Python.
  const tail = Math.max(50, Math.floor(0.2 * C.length));
  const seg = C.slice(Math.max(0, C.length - tail));

  let s = 0;
  for (let i = 0; i < seg.length; i++) s += Math.abs(seg[i]);
  return -Math.log(s / seg.length + 1e-12);
}

// ---------------------------------------------------------------------
// Bands — consequences of mu = e^(-d), not thresholds to adjust.
// ---------------------------------------------------------------------
export function band(d) {
  if (d === null || d === undefined || Number.isNaN(d)) return "N/A";
  if (d < 1.0) return "COHERENCE";
  if (d < 2.0) return "CLUTCH";
  if (d < 3.6125) return "SUB-FLOOR";
  return "VACUUM";
}

export const ANCHORS = {
  "zeta vacuum floor": 3.6125,
```

```
  "ribosome translocation": 1.88,
  "tokamak L-H": 1.93,
  "CLASH lensing masses": 1.9102,
  "ACCEPT X-ray T": 1.3702,
};

// ---------------------------------------------------------------------
// [PORT NOTE 5] Math.random() is not seedable, so nulls would differ every
// reload and could not be reproduced or audited. mulberry32 gives a seeded,
// deterministic stream. Keep the seed fixed (20250120) so a displayed z is
// reproducible by anyone checking.
// ---------------------------------------------------------------------
function mulberry32(seed) {
  let a = seed >>> 0;
  return function () {
    a |= 0; a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

function shuffled(arr, rand) {
  const a = arr.slice();
  for (let i = a.length - 1; i > 0; i--) {   // Fisher-Yates
    const j = Math.floor(rand() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

/**
 * Score an ordered positive sequence. Returns an object shaped for the UI.
 * Null is a permitted outcome — display it as one.
 */
export function resonanceScore(values, { nShuffle = 200, seed = 20250120 } = {})
  const v = Array.from(values, Number).filter(isFiniteNum);
  const d = suptProbe(v);
  if (d === null) {
    return { d_ij: null, band: "N/A", n: v.length,
             note: "sequence too short to score (need >= 4)" };
  }

  const rand = mulberry32(seed);
  const nulls = [];
  for (let i = 0; i < nShuffle; i++) {
    const nd = suptProbe(shuffled(v, rand));
```

```
    if (nd !== null && Number.isFinite(nd)) nulls.push(nd);
  }
  const nm = nulls.reduce((a, b) => a + b, 0) / nulls.length;
  const ns = Math.sqrt(
    nulls.reduce((a, b) => a + (b - nm) ** 2, 0) / nulls.length
  );
  const z = (d - nm) / (ns + 1e-12);

  const notes = [];
  const short = v.length < 50;
  if (short) {
    notes.push("short window: N < 50, tail rule spans nearly the whole " +
               "accumulator; regime-valid, low decimal precision");
  }
  if (Math.abs(z) < 2) notes.push("not separated from shuffle — read as null");
  if (d >= 1.88 && d <= 1.96) {
    notes.push("in the CLUTCH cusp; check whether the source distribution is " +
               "heavy-tailed, which can reach this band on its own");
  }

  return {
    d_ij: +d.toFixed(4),
    band: band(d),
    n: v.length,
    null_mean: +nm.toFixed(4),
    null_sd: +ns.toFixed(4),
    z: +z.toFixed(2),
    separated: Math.abs(z) >= 3,
    short_window: short,
    note: notes.join("; "),
  };
}

// ---------------------------------------------------------------------
// Feed adapters — the Sentinel's existing sequences, unmodified.
// ---------------------------------------------------------------------
const USGS = (mag, period) =>
  `https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/${mag}_${period}.geo

export async function usgsSequences({ period = "day", mag = "2.5" } = {}) {
  const js = await (await fetch(USGS(mag, period))).json();
  const rows = (js.features || [])
    .filter((f) => f.properties && f.properties.time != null)
    .map((f) => ({
      t: f.properties.time / 1000,
      m: f.properties.mag,
      z: (f.geometry && f.geometry.coordinates && f.geometry.coordinates[2]) ?? n
```

```
    }))
    .sort((a, b) => a.t - b.t);
  if (rows.length < 5) return {};

  const inter = [];
  for (let i = 1; i < rows.length; i++) {
    const dt = rows[i].t - rows[i - 1].t;
    if (dt > 0) inter.push(dt);
  }
  let mags = rows.map((r) => r.m).filter(isFiniteNum);
  if (mags.length) {
    const mn = Math.min(...mags);
    mags = mags.map((v) => v - mn + 1e-3);   // magnitudes can be negative
  }
  const depths = rows.map((r) => r.z).filter((v) => isFiniteNum(v) && v > 0);

  const out = {};
  if (inter.length >= 4) out.usgs_interevent_times = inter;
  if (mags.length >= 4) out.usgs_magnitudes = mags;
  if (depths.length >= 4) out.usgs_depths = depths;
  return out;
}

// ---------------------------------------------------------------------
// SELF-TEST — reference vectors from the Python implementation.
// ---------------------------------------------------------------------
const TEST_VECTORS = [
  { name: "t1_odd5",  input: [1, 2, 3, 5, 8],                d_ij: 0.02511321352
  { name: "t2_even6", input: [1, 2, 3, 5, 8, 13],             d_ij: 0.17841458194
  { name: "t3_n60",   input: Array.from({ length: 60 }, (_, i) => ((i * i) % 37)
                     d_ij: 0.668092754488404 },
  { name: "t5_flat",  input: [7, 7, 7, 7, 7, 7, 7, 7, 7.000001],
                     d_ij: -1.000088900581841e-12 },
];

export function selfTest({ verbose = true } = {}) {
  let pass = 0;
  for (const tv of TEST_VECTORS) {
    const got = suptProbe(tv.input);
    const ok = Math.abs(got - tv.d_ij) < 1e-12;
    if (ok) pass++;
    if (verbose) {
      console.log(
        `${ok ? "PASS" : "FAIL"}  ${tv.name.padEnd(10)} ` +
        `expected ${tv.d_ij}  got ${got}  delta ${Math.abs(got - tv.d_ij).toExpon
      );
    }
```

```
  }
  const allPass = pass === TEST_VECTORS.length;
  if (verbose) {
    console.log(
      allPass
        ? `\nAll ${pass} vectors match to 1e-12. Port is faithful.`
        : `\n${pass}/${TEST_VECTORS.length} passed. PORT IS WRONG — do not ship.`
    );
  }
  return allPass;
}

// node supt-resonance.js
if (typeof process !== "undefined" && process.argv?.[1]?.endsWith("supt-resonance
  selfTest();
}
```